

25-junio-2026

En este práctico utilizaremos **Neo4j**, un gestor de bases de datos de grafos, una tecnología diseñada para gestionar conjuntos de datos que están estructurados en forma de nodos (entidades) y aristas (relaciones entre entidades), pero donde se prioriza la conectividad y donde se requiere alto rendimiento en consultas complejas sobre relaciones entre nodos.

Parte I: Instalación

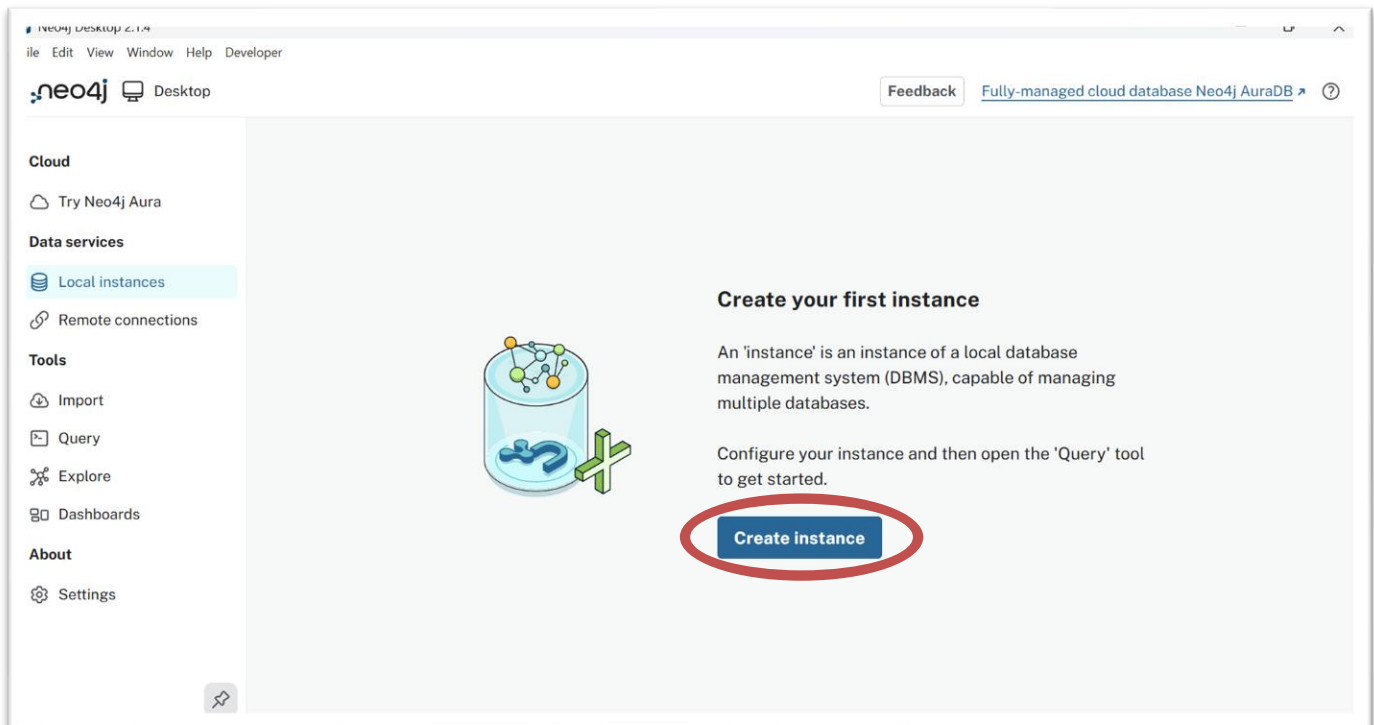
Como primer paso, **instalaremos Neo4j Desktop**, siguiendo los pasos descritos en el archivo **installNeo4jDesktop.md**, que contiene las instrucciones de instalación para Linux, Windows y macOS.

Después de instalar Neo4j Desktop, debe **levantar la base de datos** de ejemplo, llamada Movies.

Parte II: Estructura de base de datos de ejemplo en Neo4j.

La base de datos **Movies** de **Neo4j**, es una base de datos de ejemplo que contiene información sobre películas, actores, directores y otras personas relacionadas con la industria del cine. Está diseñada para mostrar las capacidades de modelado y consulta de grafos que ofrece Neo4j.

Para tener acceso a la base de datos Movies, primero debemos crear una instancia, llamémosla Movies, recuerden la contraseña ya que será útil al momento de integrar Python. Hay que esperar que termine de instalar todo lo necesario.



INFO209 – Taller de Bases de Datos – CQL
25-junio-2026

neo4j desktop 2.1.4

File Edit View Window Help Developer

Create Instance

Instance details

Instance name

Instance names must be unique

Neo4j version

Create database user

Database user

Password

Password must be at least 8 characters long

☒ Load from .dump, .backup or .tar (optional)

Cancel Create

neo4j desktop 2.1.4

File Edit View Window Help Developer

neo4j Desktop

Feedback [Fully-managed cloud database Neo4j AuraDB](#)

Search

Created Create instance

Cloud

- Try Neo4j Aura

Data services

- Local instances
- Remote connections

Tools

- Import
- Query
- Explore
- Dashboards

About

- Settings

Movies

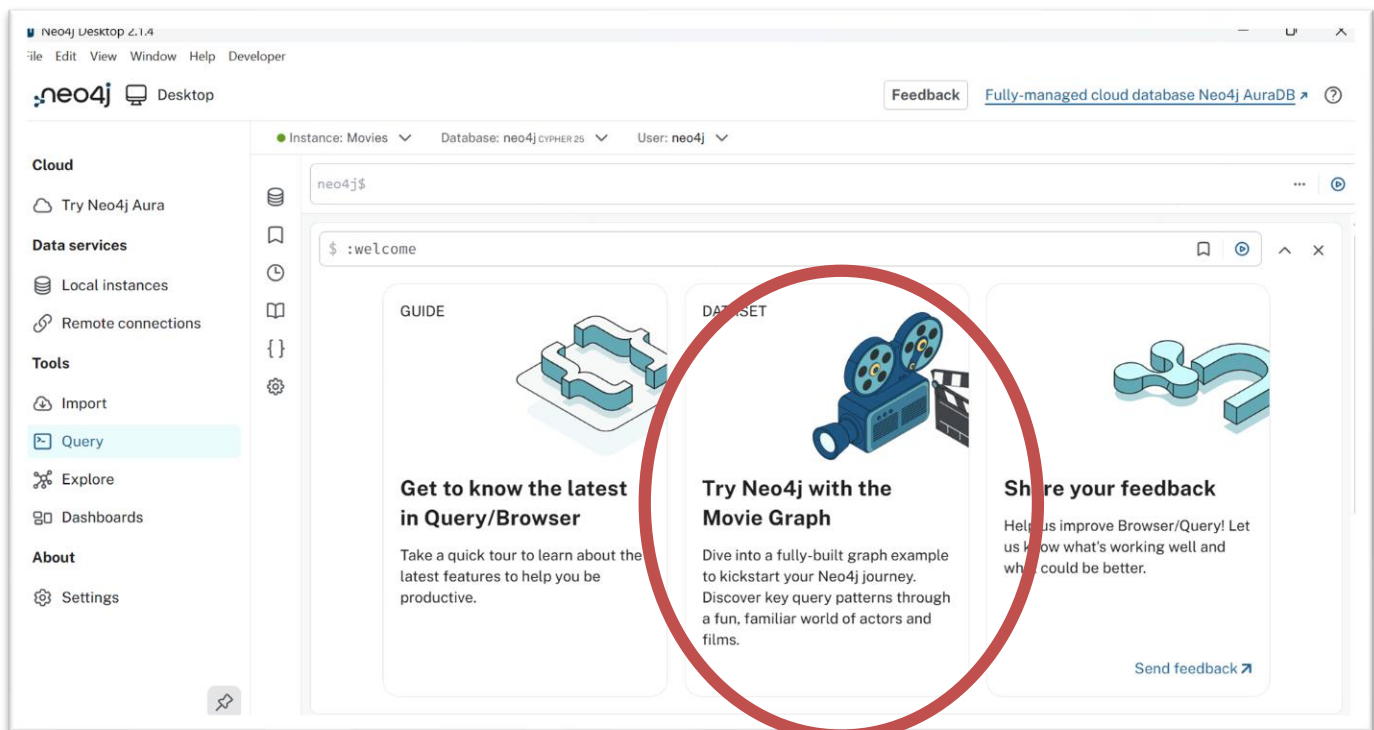
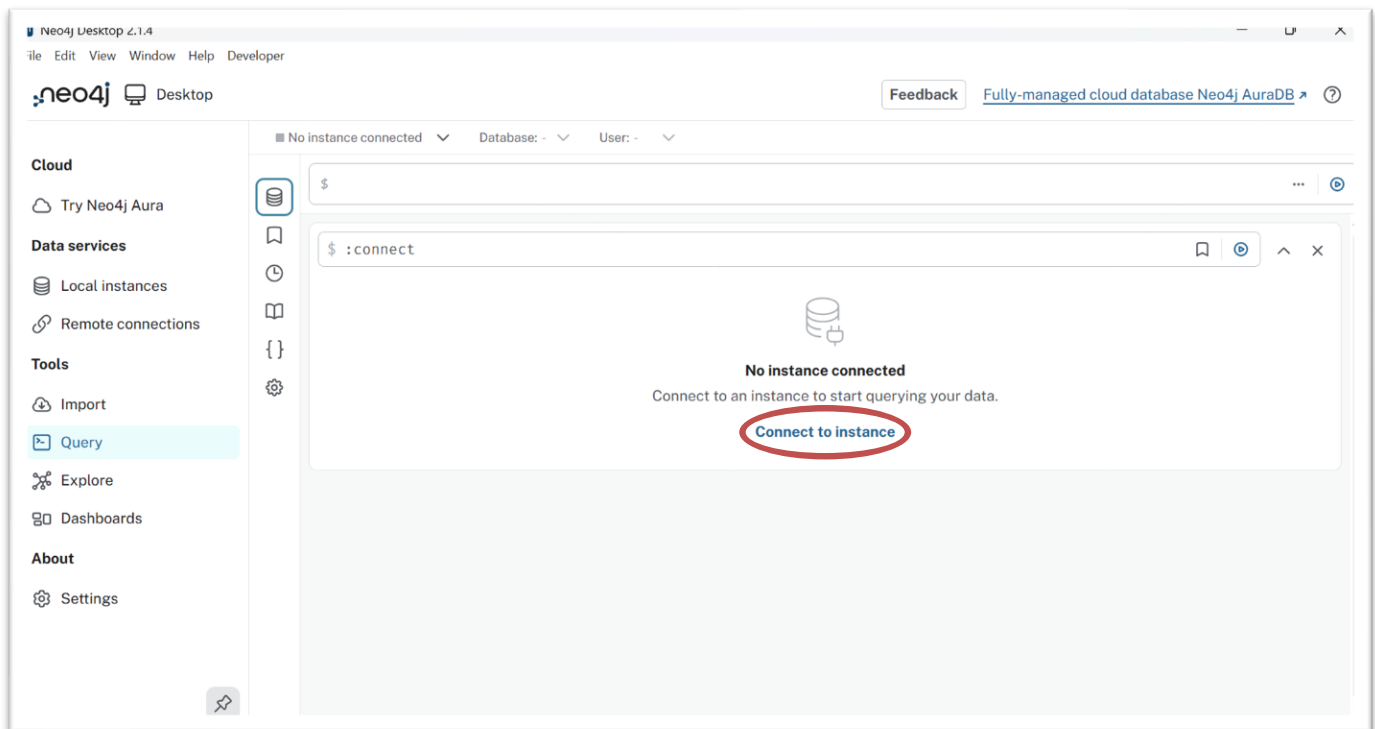
RUNNING

ID: 18d96a1d-3e5d-4ab2-86eb-6fe8364852ae | Version: 2026.05.0 | Connection URI: neo4j://127.0.0.1:7687

Path: C:\Users\sebm\Neo4j\Desktop2\Data\bmss\dbms-18d96a1d-3e5d-4ab2-86eb-6fe8364852ae | Open folder

Databases (0) Create database

INFO209 – Taller de Bases de Datos – CQL
25-junio-2026



INFO209 – Taller de Bases de Datos – CQL
25-junio-2026

Neo4j Desktop 2.1.4
File Edit View Window Help Developer

neo4j Desktop

Movie graph

Step 1/8
Movies graph

This guide uses pop-cultural connections to teach you how to use Cypher to create and query data. You will explore connections between actors, directors, and movies, and also solve the famous Bacon-path.

Note that this guide assumes that you use an empty database as you will modify it along the way. You will learn how to delete the data you create towards the end of the guide as well.

What you will learn

This guide takes around 10 minutes and by the end of it you will be able to:

- Create nodes and relationships that connect them.
- Find individual nodes by using conditional

Next

Neo4j Desktop 2.1.4
File Edit View Window Help Developer

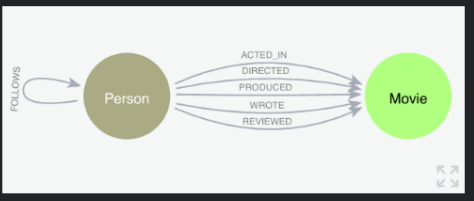
neo4j Desktop

Movie graph

Create the data

The Cypher `CREATE` clause is used to create data in your graph.

You are going to create a dataset with `Person` and `Movie` nodes and different types of relationships to connect them.

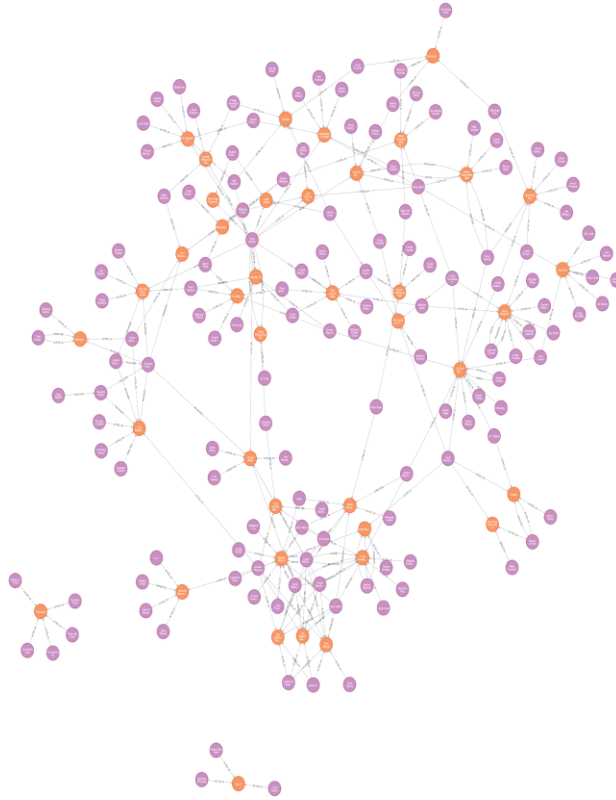


```
CREATE CONSTRAINT movie_title IF NOT EXISTS
CREATE CONSTRAINT person_name IF NOT EXISTS
```

Previous **Next**

¡LISTO! Ya tendrán el dataset de Movies cargado, ahora a seguir con el practico.

Diagrama de la base de datos:



Estructura general del modelo de datos:

- **Nodos principales:** Son descritos por el nombre de la entidad y sus atributos.
 - (:Movie {title, released, tagline})
 - (:Person {name, born})
- **Relaciones:** Representan un **vínculo dirigido** entre dos nodos, que puede tener un **tipo de relación** y también, de forma opcional, propiedades adicionales (roles).
 - (:Person)-[:ACTED_IN {roles: [...]}]->(:Movie)
 - (:Person)-[:DIRECTED]->(:Movie)
 - (:Person)-[:PRODUCED]->(:Movie)
 - (:Person)-[:WROTE]->(:Movie) (*menos frecuente*)

(Evaluado). Para mostrar que la BD Movies está completa, ejecute la siguiente Query:

○ MATCH (p) -[r]->(m) RETURN p, r, m

Parte III: Operaciones sobre la base de datos.

En esta sección, aprenderemos a utilizar el lenguaje Cypher, que permite realizar consultas, inserciones, actualizaciones y borrado de información.

I. Consultas básicas sobre nodos

1. **Obtener todos los títulos de películas registrados:**

```
MATCH (m:Movie)
RETURN m.title
```

2. **Encontrar todos los actores nacidos después de 1970:**

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
WHERE p.born > 1970
RETURN DISTINCT p
```

3. **(Evaluado) Encontrar todos los directores nacidos antes de 1940 (completar):**

```
MATCH
WHERE
RETURN DISTINCT p
```

II. Consultas sobre relaciones

4. **Mostrar los productores de todas las películas que contengan “Matrix” en el título:**

```
MATCH (p:Person)-[:PRODUCED]->(m:Movie)
WHERE m.title CONTAINS 'Matrix'
RETURN p.name, m.title
```

5. **(Evaluado) Mostrar los actores y actrices de la película Top Gun (completar):**

```
MATCH
WHERE
RETURN p
```

III. Consultas con propiedades en relaciones

6. **Listar todos los personajes (roles) interpretados por Keanu Reeves:**

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
WHERE p.name: 'Keanu Reeves'
RETURN m.title, r.roles
```

IV. Consultas combinadas y filtros

7. Listar todas las películas donde hayan actuado juntos Keanu Reeves y Carrie-Anne Moss

```
MATCH (p1:Person)-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(p2:Person)
WHERE p1.name = 'Keanu Reeves' and p2.name = 'Carrie-Anne Moss'
RETURN m.title
```

8. (Evaluado) ¿Qué actores han trabajado en películas dirigidas por Lana Wachowski?
(completar)

```
MATCH
WHERE
return DISTINCT p1.name
```

9. ¿Qué personas tienen múltiples roles en una misma película (por ejemplo, actor y guionista, o actor y director)?

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
MATCH (p) -[:WROTE|DIRECTED|PRODUCED]->(m)
RETURN DISTINCT p.name, m.title
```

V. Consultas de conteo y agrupación

10. Contar en cuántas películas ha actuado cada persona:

```
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
RETURN p.name, COUNT(m) AS peliculas_actuadas
ORDER BY peliculas_actuadas DESC
```

11. (Evaluado) ¿Quiénes han participado como director y actor en alguna película (en cuál)?
(Completar)

```
MATCH
WHERE
RETURN
```

VI. Ejercicios: INSERT y DELETE con Audrey Hepburn

Recordemos la estructura de la Base de Datos:

- **Nodos:** Películas y Personas.
 - (:Movie {title, released, tagline})
 - (:Person {name, born})
- **Relaciones:** Actuó, Dirigió, Produjo y Escribió.
 - (:Person)-[:ACTED_IN {roles: [...]}]->(:Movie)
 - (:Person)-[:DIRECTED]->(:Movie)
 - (:Person)-[:PRODUCED]->(:Movie)
 - (:Person)-[:WROTE]->(:Movie) (*menos frecuente*)

12. Insertar una nueva película

Objetivo: Crear una nueva película llamada *Breakfast at Tiffany's*, lanzada en 1961.

```
CREATE (:Movie {title: "Breakfast at Tiffany's", released: 1961, tagline: "Audrey Hepburn plays that daring, darling Holly Golightly!"})
```

13. Insertar una nueva persona

Objetivo: Crear un nodo de tipo **Person** llamado *Audrey Hepburn*, nacida en 1929.
(Completar)

```
CREATE (:Person {name: "Audrey Hepburn", born: 1929})
```

14. Crear una relación entre Audrey Hepburn y la película

Objetivo: Indicar que *Audrey Hepburn* actuó en *Breakfast at Tiffany's* como *Holly Golightly*.

```
MATCH (p:Person {name: 'Audrey Hepburn'}), (m:Movie {title: "Breakfast at Tiffany's"})
CREATE (p)-[:ACTED_IN {roles: ['Holly Golightly']}]>(m)
```

15. (Evaluado) Verificar la relación (realizar una consulta que retorne sólo los datos ingresados en los pasos 13, 14 y 15). (Completar)

```
MATCH
WHERE
return p, r, m
```


16. Eliminar solo la relación entre Audrey Hepburn y la película (no eliminar los nodos).

```
MATCH (p:Person {name: 'Audrey Hepburn'})-[r:ACTED_IN]->(m:Movie {title: "Breakfast at Tiffany's"})  
DELETE r
```

17. Eliminar la película (si ya no tiene relaciones)

```
MATCH (m:Movie {title: "Breakfast at Tiffany's"})  
DELETE m
```

De esta manera se eliminan también las relaciones, de existir:

```
MATCH (m:Movie {title: "Breakfast at Tiffany's"})  
DETACH DELETE m
```

18. Eliminar completamente a Audrey Hepburn del grafo (completar)

```
MATCH (p:Person {name:"Audrey Hepburn"})  
DELETE p
```

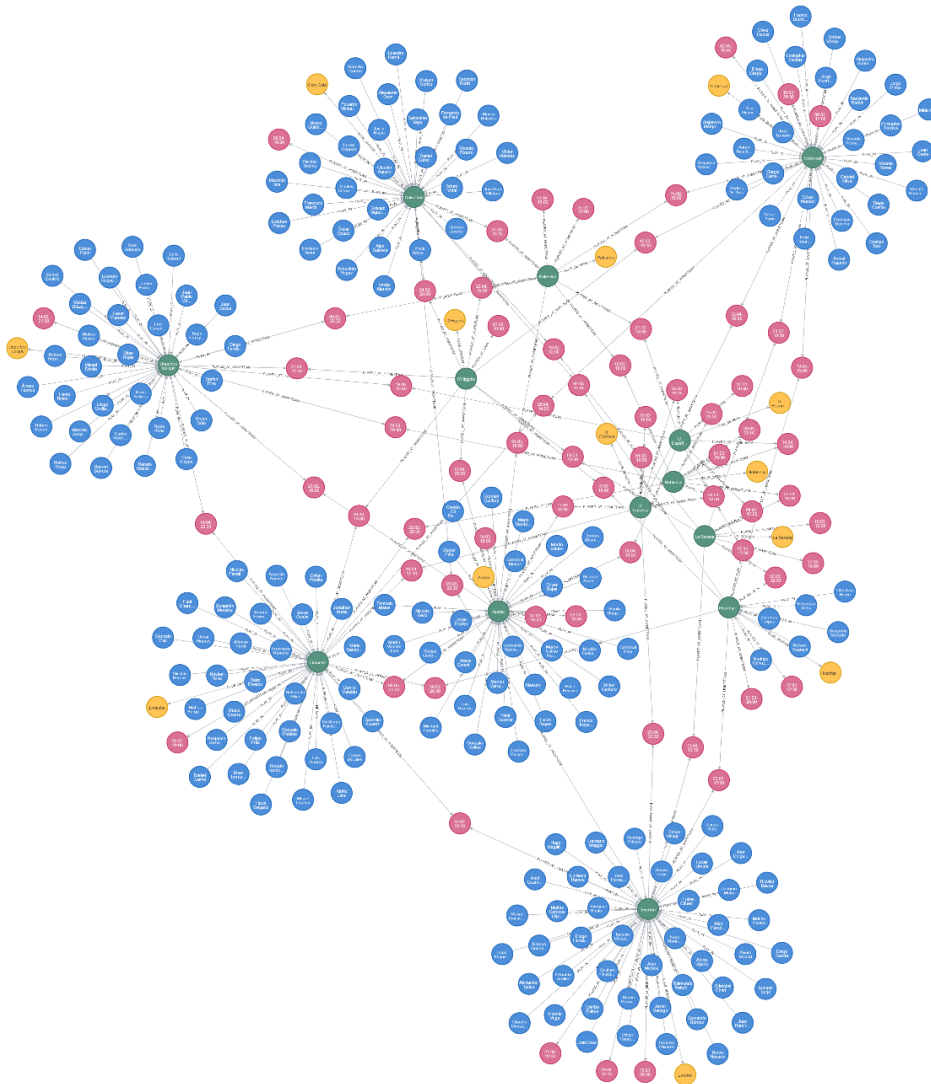
25-junio-2026

Parte IV: Base de datos de Fútbol Profesional Chileno en Neo4j.

En esta sección del laboratorio trabajaremos con un modelo de base de datos orientado a grafos, desarrollado con **Neo4j** y basado en el **fútbol chileno profesional**. El esquema fue construido con datos reales de clubes, jugadores, temporadas y partidos oficiales, correspondientes a las temporadas 2024 y 2025.

Este modelo representa las siguientes entidades: Club, FPlayer (jugadores), Season (temporadas) y MatchDay (fechas de partidos), junto con sus respectivas relaciones. Es una oportunidad ideal para **aprender Cypher** (el lenguaje de consulta de Neo4j), ya que se parte desde un dominio familiar y muy visual como el del deporte.

Diagrama de la base de datos a crear:



Entidades y relaciones por crear:

Entidades:

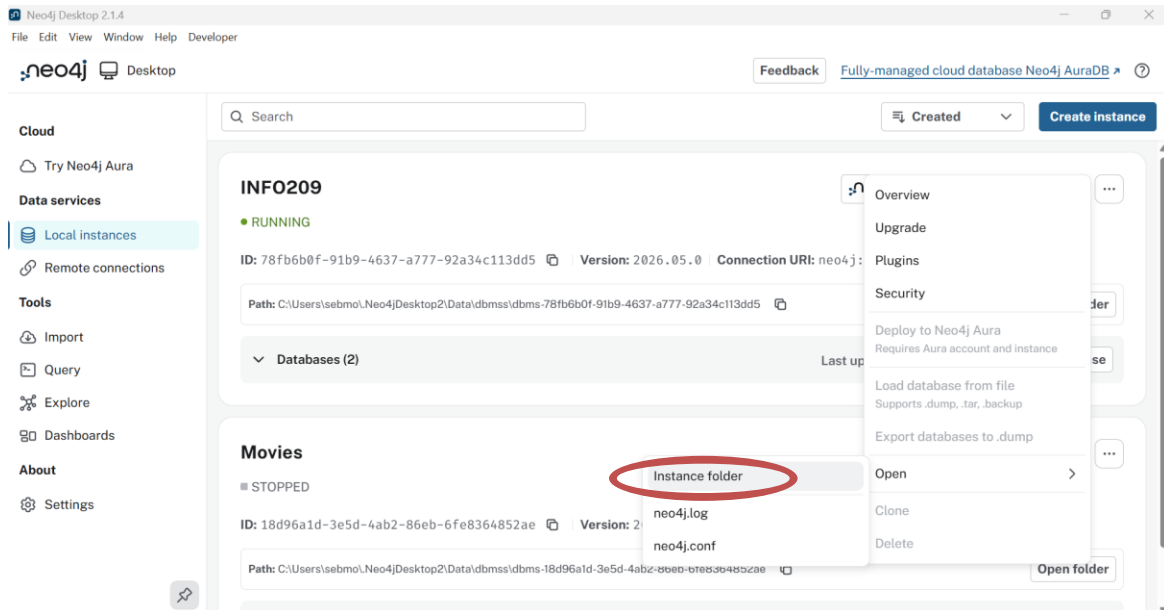
- (:Club) → Información institucional de los equipos
- (:FPlayer) → Jugadores con sus estadísticas individuales por temporada
- (:Season) → Tabla de posiciones por año
- (:MatchDay) → Información detallada de cada partido jugado

Relaciones principales:

(:Club)-[:CLUB_SEASON]->(:Season)	Club participó en esa temporada
(:Club)-[:PLAYED_AT_AWAYTEAM]->(:MatchDay)	Club jugó como visita en ese partido
(:Club)-[:PLAYED_AT_HOMETEAM]->(:MatchDay)	Club jugó como local en ese partido
(:FPlayer)-[:PLAYS_FOR]->(:Club)	Jugador juega en ese club

Pasos para crear la base de datos

- Descargue los archivos CSV desde SiveducMD
- En Neo4j, cree una nueva instancia llamada INFO209 (recuerde la contraseña, es importante)
- Entre a las configuraciones de la instancia → Open → Instance Folder. Dentro de la carpeta import, copiar todos los CSV entregados



- Conéctese a dicha instancia desde la herramienta Query e introducir uno a uno, los comandos disponibles en el archivo comandosBd.md, disponible en SiveducMD
- **(Evaluado)** Verificar creación de la base de datos:
 - MATCH (n) -[r]->(b) RETURN n, r, b
 - ¿Qué hace este comando?

I. Consultas

- 1) Mostrar cuáles son los atributos de la entidad Club.

```
MATCH (n:Club) Return n
```

```
Dt, Facebook, Instagram, youtube, name, president, stadium, yearoffundation
```

- 2) (Evaluado) Listar todos los clubes creados, con su nombre y presidente.

```
MATCH  
RETURN
```

- 3) Listar todos los jugadores de Colo Colo, para la temporada 2025, indicando nombre, posición y edad.

```
MATCH (j:FPlayer) -[r:PLAYS_FOR]-> (c:Club)  
WHERE r.year = 2025 and c.name = "Colo Colo"  
return j.name, j.position, j.age
```

- 4) Listar los nombres de todos los clubes, indicando cuántos partidos ha jugado cada uno como visita.

```
MATCH (c:Club) -[:PLAYED_AT_AWAYTEAM]-> (m:MatchDay)  
return c.name, count(m) as cantidad_partidos_visita
```

- 5) Listar los 5 jugadores con más goles, en orden descendente.

```
i. MATCH (j:FPlayer)  
WHERE j.goals IS NOT NULL  
return j.name, j.goals as goles  
ORDER BY goles DESC  
LIMIT 5
```

- 6) (Evaluado) Listar el nombre del club y el nombre del dt, para aquellos clubes que convirtieron más de 3 goles como local en el mismo partido.

```
MATCH  
WHERE  
RETURN DISTINCT
```

- 7) (Evaluado) Genere el grafo de partidos y equipos (local y visita), para aquellos partidos en que los clubes convirtieron más de 3 goles como local. ¿Hay algún equipo que lo haya logrado más de una vez?

```
MATCH  
WHERE  
RETURN
```

Parte V: Acceso a la base de datos desde un programa externo

Es posible realizar consultas y otras operaciones sobre una base de datos Neo4j, desde un programa externo.

En este curso, veremos cómo acceder a nuestra base de datos desde un programa escrito en Python. Para el ejemplo que realizaremos en esta clase, debemos tener instalado lo siguiente:

- Python
- Algún entorno de programación, como por ejemplo VS Code.
- Las bibliotecas pandas y matplotlib. Si no las tiene, ejecute `pip install` desde PowerShell (caso Windows).
- La biblioteca neo4j (se instala también con `pip install`).

Adicionalmente, **debemos tener un usuario creado en la base de datos**, y debemos tener **levantada la base de datos** correspondiente en nuestro servidor Neo4j.

Este es un ejemplo de programa Python que **genera un gráfico** en base a los resultados de una consulta realizada a una base de datos Neo4j, que retorna cuántos roles distintos ha ejercido cada persona en diferentes películas (actor, director, productor, escritor). Esta es la consulta.

```
MATCH (p:Person)-[r]->(:Movie)
WHERE type(r) IN ['ACTED_IN', 'DIRECTED', 'PRODUCED', 'WROTE']
WITH p.name AS person, collect(DISTINCT type(r)) AS roles
RETURN person, size(roles) AS num_roles
ORDER BY num_roles DESC
LIMIT 10
```

El código Python se encarga de importar las bibliotecas, conectarse a la base de datos, realizar la consulta y graficar la salida.

```
from neo4j import GraphDatabase
import pandas as pd

# Conexión con la base de datos
URI = "neo4j://localhost:7687"
USER = "jpsf1971"
PASSWORD = "jpsf1971" # Cambia esto por tu usuario y contraseña

driver = GraphDatabase.driver(URI, auth=(USER, PASSWORD))
driver.verify_connectivity()

# Función para ejecutar consultas
def run_query(cypher_query: str, parameters: dict = {}) -> pd.DataFrame:
    with driver.session() as session:
        result = session.run(cypher_query, parameters)
        # Se convierte a DataFrame
        data = [record.data() for record in result]
        return pd.DataFrame(data)
```

```
# ----- EJEMPLO DE CONSULTA ----- #
query_multiples_roles = """
MATCH (p:Person)-[r]->(:Movie)
WHERE type(r) IN ['ACTED_IN', 'DIRECTED', 'PRODUCED', 'WROTE']
WITH p.name AS person, collect(DISTINCT type(r)) AS roles
RETURN person, size(roles) AS num_roles
ORDER BY num_roles DESC
LIMIT 10
"""

df_personas = run_query(query_multiples_roles)
print(df_personas)

# ----- EJEMPLO DE VISUALIZACIÓN ----- #
import matplotlib.pyplot as plt

def plot_personas(df):
    if df.empty:
        print("No se encontraron datos.")
        return
    plt.figure(figsize=(10, 6))
    plt.barh(df['person'], df['num_roles'], color='skyblue')
    plt.xlabel("Cantidad de roles")
    plt.title("Personas")
    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.show()

plot_personas(df_personas)

# Cierre de la conexión
driver.close()
```

1) (Evaluado) Usando como base el ejemplo anterior, genere un gráfico que muestre, para cada club, cuántos jugadores tiene que sean mayores de 25 años